

SAL Engineering & Technical Institute

CE & CSE Department

Analysis and Design of Algorithms (BE04000241)

ASSIGNMENT – 2 (CO2 & CO3)

Q1. Binary Search using Divide and Conquer

Binary Search is an efficient algorithm to find an element in a sorted array. It uses the Divide and Conquer strategy to repeatedly halve the search space.

Divide and Conquer Concept

- (i) Divide: Split the array into two halves using the middle index.
- (ii) Conquer: Compare target with the middle element.
 - If equal → element found.
 - If smaller → search left half.
 - If larger → search right half.
- (iii) Combine: No combining needed — just return the result.

Algorithm (Steps)

- (i) Set low = 0, high = $n - 1$.
- (ii) While low $\leq$ high:
 - Calculate mid = $(\text{low} + \text{high}) / 2$
 - If $\text{array}[\text{mid}] == \text{key}$ → return position
 - If $\text{key} < \text{array}[\text{mid}]$ → set high = mid - 1
 - Else → set low = mid + 1
- (iii) If loop ends → element not found (return -1).

Example

Sorted array: [10, 20, 30, 40, 50, 60, 70] → Find 50

Step	low	high	mid (value)
1	0	6	3 → 40
2	4	6	5 → 60
3	4	4	4 → 50 ✓

Result: Element 50 found at index 4.

Time Complexity: $O(\log n)$ | Space Complexity: $O(1)$

Q2. Merge Sort Algorithm with Example

Merge Sort is a stable, efficient sorting algorithm that uses the Divide and Conquer approach. It repeatedly splits the array in half, sorts each half, and merges them back together.

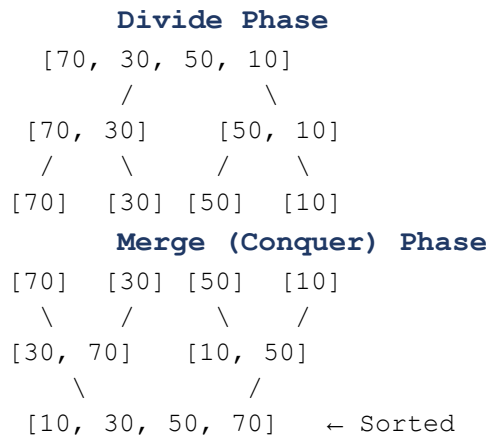
Three Main Steps

- (i) Divide: Split the array into two halves repeatedly until each sub-array has one element.
- (ii) Conquer: A single element is already sorted.
- (iii) Combine (Merge): Merge the sorted halves to produce a fully sorted array.

Algorithm (Steps)

- (i) If the array has one element, return it.
- (ii) Find the middle of the array.
- (iii) Divide into left and right halves.
- (iv) Apply Merge Sort on both halves.
- (v) Merge the two sorted halves.

Example — Unsorted Array: [70, 30, 50, 10]



Property	Value
Time Complexity (all cases)	$O(n \log n)$
Space Complexity	$O(n)$
Stable Sort?	Yes

Q3. Making Change Problem using Dynamic Programming

The Making Change problem involves finding the minimum number of coins from a given set of denominations to make a target amount.

DP Idea

Let $dp[i]$ = minimum coins needed to make amount i .

- Base Case: $dp[0] = 0$
- Formula: $dp[i] = \min(dp[i], dp[i - \text{coin}] + 1)$ for all coins where $i \geq \text{coin}$

Example: Coins = {1, 3, 4}, Amount = 6

Amount i	0	1	2	3	4 (use 4)	5 (use 1+4)
$dp[i]$	0	1 (1)	2 (1+1)	1 (3)	1 (4)	2 (1+4)

Amount	$dp[6]$
6 (use 3+3 or 4+1+1)	2 coins

Answer: 2 coins (e.g., 3 + 3)

Algorithm (Steps)

- (i) Create array $dp[0 \dots \text{amount}]$, set $dp[0] = 0$, all others = Infinity.
- (ii) For $i = 1$ to amount:
- For each coin c :
 - If $i \geq c \rightarrow \text{update } dp[i] = \min(dp[i], dp[i - c] + 1)$
- (iii) $dp[\text{amount}]$ is the final answer.
- Time Complexity: $O(\text{amount} \times \text{number_of_coins})$

Q4. Longest Common Subsequence (LCS) Problem

The LCS problem finds the longest sequence of characters that appears in both strings in the same order, but not necessarily continuously.

A subsequence is characters appearing in order but not necessarily adjacent. For example, 'ACE' is a subsequence of 'ABCDE'.

DP Idea

Let $dp[i][j]$ = length of LCS of first i characters of String 1 and first j characters of String 2.

- If characters match: $dp[i][j] = dp[i-1][j-1] + 1$
- If characters don't match: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

Example: String 1 = ABCDGH, String 2 = AEDFHR

Common subsequences: ADH, ADFH $\rightarrow$ LCS = ADH (length = 3)

		A	E	D	F	H	R	
A B C	0	0	0	0	0	0	0	
	0	1	1	1	1	1	1	
	0	1	1	1	1	1	1	
	0	1	1	1	1	1	1	
D G	0	1	1	2	2	2	2	
	0	1	1	2	2	2	2	
H	0	1	1	2	2	3	3	

LCS Length = 3 (ADH) | Time Complexity: $O(m \times n)$ | Space: $O(m \times n)$

ASSIGNMENT – 3 (CO3 & CO4)

Q1. Job Scheduling Problem using Greedy Approach

The Job Scheduling Problem (also called Job Sequencing with Deadlines) involves scheduling jobs to maximize total profit. Each job has a deadline and a profit, and takes 1 unit of time.

Greedy Strategy

- Sort jobs in descending order of profit.
- Pick the job with the highest profit first.
- Assign it to the latest available time slot before its deadline.

Algorithm (Steps)

- Sort jobs in descending order of profit.
- Create a time slot array (size = max deadline).
- For each job, try to place it in the latest available slot before its deadline.
- If slot is free → assign job. Else → skip job.

Example

Job	Deadline	Profit	Sorted Order
J1	2	100	1st (100)
J3	2	27	2nd (27)
J4	1	25	3rd (25)
J2	1	19	4th (19)
J5	3	15	5th (15)

Scheduling result:

- Slot 1 → J3 (latest free slot before deadline 2)
- Slot 2 → J1 (assigned to slot 2, deadline = 2)
- Slot 3 → J5 (deadline = 3)

Total Profit = 100 + 27 + 15 = 142

Time Complexity: Sorting $O(n \log n)$, Scheduling $O(n^2)$

Q2. Fractional Knapsack Problem

The Fractional Knapsack problem aims to maximize profit in a knapsack with limited capacity. Unlike 0/1 Knapsack, fractions of items can be taken.

Greedy Idea

- Calculate value/weight ratio for each item.
- Sort items in descending order of ratio.
- Take items fully or partially until the knapsack is full.

Example — Capacity = 50

Item	Weight	Value	Ratio (V/W)	Action
I1	10	60	6.0 (1st)	Take full

I2	20	100	5.0 (2nd)	Take full
I3	30	120	4.0 (3rd)	Take 20/30

Solution:

- Take I1 fully $\rightarrow 60$ (remaining capacity = 40)
- Take I2 fully $\rightarrow 100$ (remaining capacity = 20)
- Take 20/30 of I3 $\rightarrow 120 \times (20/30) = 80$

Total Value = 60 + 100 + 80 = 240

Time Complexity: $O(n \log n)$ for sorting

Q3. Topological Sorting of a Graph

Topological Sorting is a linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge ($u \rightarrow v$), vertex u comes before v . It is used in task scheduling, course prerequisites, and build systems.

Two Methods

(1) Kahn's Algorithm (BFS-based)

- Compute the in-degree of all vertices.
- Initialize a queue with all vertices having in-degree = 0.
- While queue is not empty: remove a vertex, add to result, reduce in-degree of adjacent vertices by 1. If any reach 0, add to queue.
- Repeat until all vertices are processed.

(2) DFS-based Algorithm

- Mark all vertices as unvisited.
- Perform DFS for each unvisited vertex.
- After visiting all adjacent vertices, push current vertex onto a stack.
- Pop elements from the stack to get the topological order.

Example — Graph Edges: $A \rightarrow C$, $B \rightarrow C$, $C \rightarrow D$

Vertex	In-Degree	Processing Order
A	0	1st (start)
B	0	2nd (start)
C	$2 \rightarrow 0$	3rd
D	$1 \rightarrow 0$	4th

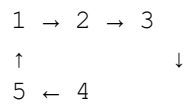
Topological Order: A, B, C, D (or B, A, C, D)

Time Complexity: $O(V + E)$ where V = vertices, E = edges

Q4. Strongly Connected Components (SCC)

In a directed graph, a Strongly Connected Component (SCC) is a group of vertices where every vertex is reachable from every other vertex in that group. For any two vertices u and v in an SCC, there exists a path $u \rightarrow v$ and also $v \rightarrow u$.

Example



{1, 2, 3, 4, 5} form one SCC — all are mutually reachable.

Algorithms to Find SCCs

- Kosaraju's Algorithm → Uses two DFS passes.
- Tarjan's Algorithm → Uses single DFS with a stack.
- Gabow's Algorithm → Another single-DFS approach.

Time Complexity: $O(V + E)$ for all SCC algorithms

Applications

- Detecting cycles in directed graphs.
- Program flow analysis.
- Deadlock detection in operating systems.
- Social network analysis.

ASSIGNMENT – 4 (CO5)

Q1. Sum of Subsets Problem using Backtracking

The Sum of Subsets problem finds all subsets of a given set of numbers whose sum equals a specified target value.

Concept of Backtracking

Backtracking is a technique where we build a solution step-by-step and backtrack (undo choices) when a solution is not feasible.

Approach

At each step, decide whether to include or exclude the current element. Continue recursively until:

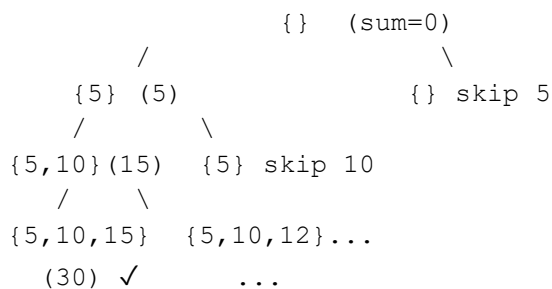
- Sum equals target → solution found (print subset).
- Sum exceeds target → backtrack (stop exploring that path).

Algorithm (Steps)

- (i) Start with an empty subset.
- (ii) For each element: include it and recurse, then exclude it and recurse.
- (iii) If current sum = target → print subset.
- (iv) If current sum > target → stop exploring that path.

Example: Set = {5, 10, 12, 13, 15, 18}, Target = 30

Backtracking Tree (partial):



Possible Solutions: {5, 10, 15} and {12, 18}

Time Complexity: $O(2^n)$ worst case

Q2. Knapsack Problem using Branch and Bound

The 0/1 Knapsack problem: select items such that total weight $\leq$ capacity and total profit is maximized. Each item is either taken or not (no fractions).

Branch and Bound Concept

- Branch: Divide into subproblems (include item / exclude item).
- Bound: Calculate an upper bound (best possible profit) for each node.
- If bound is worse than current best → prune (discard that branch).

Approach

- Use a state-space tree where each node = a decision.
- Use the Fractional Knapsack idea to compute the upper bound.
- Expand nodes with the best bound first (using a priority queue).
- Ignore nodes whose bound < current best profit.

Example — Capacity = 6

Item	Weight	Profit	Ratio (P/W)
1	2	40	20
2	3	50	16.7
3	5	100	20

Best Solution: Select Item 1 + Item 2 (weight = 5 ≤ 6)

Total Profit = 40 + 50 = 90

Time Complexity: $O(2^n)$ worst case, but much faster in practice due to pruning.

Branch and Bound reduces unnecessary computations and efficiently finds the optimal solution.

Q3. NP-Complete Problems with Examples

NP-Complete problems are problems that are: (i) in NP (Non-deterministic Polynomial time), and (ii) as hard as any other problem in NP.

Key Concepts

- A problem is in NP if its solution can be verified in polynomial time.
- NP-Complete problems are the hardest problems in NP.
- If one NP-Complete problem is solved in polynomial time → all NP problems can be solved in polynomial time.
- No known polynomial-time solution exists (as of now).

Characteristics

- Easy to verify a solution.
- Hard to find a solution.
- Usually solved using backtracking, branch and bound, approximation, or heuristics.

Examples of NP-Complete Problems

Problem	Description
Travelling Salesman (TSP)	Find shortest tour visiting all cities exactly once.
0/1 Knapsack	Select items for max profit without exceeding capacity.
Subset Sum	Determine if a subset exists with a given sum.
Graph Coloring	Color graph vertices so no two adjacent share a color.
Hamiltonian Cycle	Find a cycle that visits each vertex exactly once.
Boolean SAT	Determine if a Boolean formula can be satisfied.

Q4. Differentiate between NP-Hard and NP-Complete Problems

Basis	NP-Hard	NP-Complete
Definition	At least as hard as any problem in NP.	Both NP and NP-Hard.

Belongs to NP	Not necessarily.	Always belongs to NP.
Verification	May not be verifiable in polynomial time.	Verifiable in polynomial time.
Type of Problems	Decision or Optimization problems.	Only Decision problems.
Complexity Level	Equal or harder than NP problems.	Hardest problems within NP.
Solution Existence	May or may not have efficient solution.	No known polynomial-time solution.
Examples	Halting Problem, TSP (optimization version)	SAT, Subset Sum, Graph Coloring

Key Insight: Every NP-Complete problem is NP-Hard, but not every NP-Hard problem is NP-Complete. NP-Hard is a superset.